

Capitolo 2

Background

Obiettivo del capitolo

Questo capitolo introduce i concetti necessari per formulare il problema della *Documentation-Driven Threat Analysis* senza anticipare come risultati le scelte progettuali che verranno formalizzate nei capitoli successivi. L'obiettivo è costruire un lessico comune e chiarire cinque premesse: l'analisi delle minacce può iniziare prima dell'implementazione; le metodologie non osservano necessariamente il sistema attraverso la stessa rappresentazione; i risultati dell'analisi devono essere distinti dai requisiti di sicurezza che ne possono derivare; traceability, provenance e versionamento condizionano la validità dell'analisi nel tempo; automazione e modelli linguistici possono assistere la costruzione di artefatti analitici, ma non eliminano la necessità di validarne semantica, copertura e provenienza.

La distinzione fra background e contributo della tesi è intenzionale. In questo capitolo non si assume che la documentazione di progetto sia già sufficiente a costruire automaticamente una rappresentazione completa del sistema, nè che una singola struttura dati sia adatta a ogni metodo di threat analysis. Il perimetro della tesi non comprende però la trasformazione di documentazione arbitraria o legacy in una forma analizzabile: i capitoli metodologici definiranno invece un contratto DDTA *portable-by-construction*, cioè le informazioni e le relazioni che devono essere documentate e governate affinché l'analizzabilità possa essere valutata. Analogamente, non si assume che una pipeline automatizzata produca finding accettabili senza revisione. Il Capitolo 3 confronterà in modo sistematico gli approcci esistenti e formulerà il research gap; il Capitolo 5 definirà invece il modello DDTA proposto per affrontarlo.

2.1 Threat modeling e analisi precoce

Il threat modeling può essere descritto come un'attività strutturata di rappresentazione del sistema, identificazione di condizioni o scenari di minaccia e valutazione delle conseguenze e delle possibili risposte di sicurezza. Per la tesi è importante separare questa attività dall'analisi del solo codice implementato. La systematic literature review di Tuma et al. (2018), che classifica ventisei tecniche di threat analysis rappresentate da trentotto studi primari, colloca la maggior parte delle tecniche esaminate nelle fasi precedenti all'implementazione: requisiti, architettura e design sono livelli di applicazione ricorrenti, mentre il codice sorgente compare raramente come prerequisito. Requisiti, descrizioni testuali, obiettivi, scenari e modelli architetturali sono invece input frequenti.

Questo dato non dimostra che un insieme arbitrario di documenti sia già un threat model. Mostra però che l'analisi può essere significativa quando il software non esiste ancora, a condizione che le informazioni necessarie alla metodologia siano disponibili. Nel perimetro di questa tesi, tale distinzione conduce a un problema più circoscritto: rendere esplicito a monte quale conoscenza debba essere documentata, identificata, collegata e mantenuta affinché una rappresentazione analizzabile possa essere costruita senza incorporare la semantica di una singola metodologia.

Gli approcci requirements-oriented forniscono esempi concreti. I misuse case di Sindre and Opdahl (2005) estendono gli use case funzionali introducendo misuser, misuse case e security use case. Le relazioni di minaccia e mitigazione collegano il comportamento ostile alle funzionalità da proteggere, permettendo di lavorare prima dell'implementazione. Il metodo non parte tuttavia da testo arbitrario: richiede scenari funzionali e un'interpretazione esplicita di attori, obiettivi, asset e percorsi dannosi.

L'approccio goal-oriented di van Lamsweerde (2004) affronta lo stesso problema da una prospettiva diversa. Il modello intenzionale del sistema viene affiancato da un anti-model di attaccanti, anti-goal, capacità e vulnerabilità. Anche in questo caso l'analisi può precedere il codice, ma le affermazioni provenienti da stakeholder e documenti devono essere strutturate in goal, agenti, oggetti, operazioni, requisiti, aspettative e proprietà di dominio. La possibilità di eseguire raffinamenti formali o generare scenari dipende dalla disponibilità di tale struttura, non dal solo possesso di documenti testuali.

Haley et al. (2008) collegano i security requirements a specifici requisiti funzionali e ne discutono la soddisfazione mediante argomenti espliciti. Il sistema deve essere rappresentato attraverso contesto, domini, fenomeni condivisi e comportamento; la trasformazione dal materiale documentale a questa rappresentazione rimane esterna al framework. Whittle et al. (2008) mostrano invece che scenari ostili e mitigazioni possono essere incorporati in modelli comportamentali ed eseguiti come test a livello

di modello prima che l'implementazione esista. Anche qui la possibilità di esecuzione dipende da una rappresentazione precisa di scenari, messaggi, stati e precondizioni.

Infine, Hatebur et al. (2006) separano problema di sicurezza, requisito concretizzato, principio di soluzione, meccanismo e specifica. Questa separazione è importante perché evita di confondere l'identificazione del problema con la scelta della contromisura. Assunzioni sul dominio e sulle capacità dell'attaccante condizionano la validità del passaggio da problema a soluzione.

Nel seguito della tesi vengono quindi mantenuti distinti quattro concetti. Una **minaccia** descrive una condizione, capacità o azione in grado di compromettere una proprietà di sicurezza. Un **finding** è un risultato analitico riferito a uno specifico sistema, baseline e metodo. Una **mitigazione** è una possibile risposta al problema identificato. Un *Security Requirement* è invece un'obbligazione del prodotto, collegata al comportamento o alla proprietà che intende proteggere e, in prospettiva, verificabile. Le quattro nozioni sono correlate ma non intercambiabili.

La dimensione temporale completa il quadro. Una threat analysis non deve necessariamente essere intesa come documento prodotto una sola volta. La review di Tuma et al. (2018) include tecniche applicabili in più fasi del ciclo di sviluppo; Whittle et al. (2008) mostrano la possibilità di riutilizzare scenari ostili come regressione a livello di modello. Questa continuità rende rilevante non soltanto produrre un'analisi iniziale, ma sapere a quale stato del sistema essa si riferisse e quando un cambiamento renda necessaria una rivalutazione.

2.2 Rappresentazioni e viewpoint analitici

Una metodologia di threat analysis non opera sul “sistema” in senso astratto, ma su una sua rappresentazione. La forma di tale rappresentazione determina quali entità sono visibili, quali relazioni possono essere espresse e quali domande analitiche possono essere poste. Il corpus considerato nella tesi non supporta l'assunzione che una singola notazione o una singola vista sia universalmente sufficiente.

Negli approcci scenario-based la struttura principale è il comportamento osservabile. Nei misuse case, attori legittimi e ostili, use case, misuse case e security use case rendono espliciti percorsi desiderati e indesiderati (Sindre and Opdahl, 2005). Gli executable misuse case rendono la rappresentazione ancora più precisa: scenari funzionali, modifiche dell'attaccante e mitigazioni vengono composti, trasformati in modelli eseguibili e utilizzati per verificare tracce ostili note (Whittle et al., 2008). Questa prospettiva è adatta a esprimere ordine, interazioni e condizioni comportamentali, ma può non includere dettagli di rete, storage o piattaforma necessari per minacce di livello inferiore.

Negli approcci goal-oriented il centro dell'analisi cambia. Il primal model e l'anti-model

di van Lamsweerde (2004) rappresentano goal, anti-goal, agenti, oggetti, operazioni, vulnerabilità e capacità dell'attaccante. La decomposizione AND/OR consente di ragionare su intenzioni e condizioni di soddisfazione che non sono naturalmente espresse da una vista puramente topologica.

Gli approcci basati su assurance e problem frames aggiungono ulteriori informazioni. Haley et al. (2008) richiedono contesto, domini, fenomeni condivisi e comportamento per costruire argomenti di soddisfazione. Hatebur et al. (2006) modellano domini ambientali, interfacce, fenomeni, assunzioni e capacità dell'attaccante per mantenere distinta l'analisi del problema dalla scelta del meccanismo. Queste strutture rendono esplicite responsabilità e premesse che una rappresentazione di soli componenti e flussi potrebbe lasciare implicite.

DFD, grafi tipizzati e inventory di asset assumono invece un ruolo importante negli approcci software- e model-centric. La review di Granata and Rak (2024) mostra che DFD e rappresentazioni a grafo sono comuni nelle tecniche automatizzate e che cataloghi e regole possono essere applicati a elementi, relazioni, proprietà, tecnologie e protocolli del modello. Il risultato dell'automazione dipende quindi dal modello fornito e dalla semantica delle regole che vi operano.

L'utilità di una vista non può tuttavia essere dedotta dal solo fatto che essa sia convenzionale. Un *Data Flow Diagram* (DFD) mette principalmente in evidenza processi o componenti, data store e flussi di dati, mentre un UML *sequence diagram* rappresenta l'ordine temporale dei messaggi scambiati fra attori e componenti in uno specifico scenario. Nell'esperimento di Mbaka et al. (2025), tutti i partecipanti disponevano di scenario testuale, materiale STRIDE, descrizioni strutturate delle minacce, assunzioni e sequence diagram; soltanto il gruppo di intervento riceveva in aggiunta un DFD. Fra i partecipanti che disponevano di entrambe le rappresentazioni, il DFD ricevette un punteggio di utilità percepita significativamente superiore a quello del sequence diagram; lo studio non stabilisce le cause di questa preferenza. L'aggiunta del DFD non produsse tuttavia un miglioramento statisticamente significativo nella corretta classificazione di minacce reali o fabbricate. Il risultato non dimostra che i DFD siano generalmente inefficaci: è limitato a partecipanti novizi, due scenari e uno specifico compito di validazione. Mostra però che valore percepito e incremento misurato di correttezza sono proprietà distinte e che una rappresentazione deve essere valutata rispetto al compito concreto.

Il model-based security testing utilizza ancora un altro viewpoint. La Rapid Review di Lonetti et al. (2023) descrive UML/OCL, timed automata, Colored Petri Nets, attack tree e altre strutture arricchite con proprietà di sicurezza e criteri di coverage. Da tali modelli vengono derivati test astratti, che richiedono successivamente concretizzazione, adapter, ambiente di esecuzione e oracle per produrre evidenza sul sistema reale. Una

rappresentazione sufficiente per generare threat candidate non è quindi automaticamente sufficiente per generare o eseguire test.

La letteratura storica distingue inoltre centri di analisi risk-centric, attack-centric, software-centric, goal-oriented e security-requirements-oriented (Tuma et al., 2018). Per questa tesi tali categorie sono considerate *viewpoint analitici*: indicano quale conoscenza viene privilegiata dal metodo, non moduli necessariamente mutuamente esclusivi.

Tabella 2.1: Esempi di viewpoint analitici emersi dalle fonti considerate.

Viewpoint	Conoscenza messa in primo piano	Esempi nel corpus
Scenario / misuse	Attori, percorsi funzionali e ostili, condizioni, mitigazioni	Misuse case ed executable misuse case (Sindre and Opdahl, 2005; Whittle et al., 2008)
Goal / anti-goal	Goal, anti-goal, agenti, oggetti, vulnerabilità, capacità dell'attaccante	Anti-model goal-oriented (van Lamsweerde, 2004)
Problem / assurance	Domini, fenomeni, assunzioni, requisiti, argomenti di soddisfazione	Security requirements assurance e problem frames (Haley et al., 2008; Hatebur et al., 2006)
Software / model	Componenti, flussi, relazioni, proprietà, protocolli, regole	Automated threat modelling (Granata and Rak, 2024)
Asset-centered	Asset da proteggere, funzioni, failure mode, proprietà di sicurezza, minacce	STRIDE-AI (Mauri and Damiani, 2022)

La conclusione rilevante per il background è prudente: metodologie differenti possono richiedere viste e semantiche differenti dello stesso sistema. Non è giustificato assumere che la rappresentazione preferita da un singolo metodo costituisca, per questo solo motivo, il modello canonico dell'intero progetto.

2.3 Security Requirements Engineering

Threat analysis e Security Requirements Engineering sono strettamente collegate, ma non coincidono. Una tecnica può individuare scenari ostili, condizioni di vulnerabilità o failure mode senza avere ancora deciso quale obbligo debba essere imposto al prodotto; allo stesso modo, un requisito di sicurezza può dipendere da obiettivi, assunzioni e vincoli che richiedono un ragionamento più ampio della sola classificazione della minaccia.

I misuse case forniscono un primo esempio del passaggio fra comportamento funzionale, comportamento ostile e risposta di sicurezza. I security use case vengono collegati ai misuse case come mitigazioni, rendendo visibile la relazione con la funzionalità originaria (Sindre and Opdahl, 2005). Il vantaggio concettuale è che la risposta di

sicurezza non rimane una nota isolata: conserva il legame con il comportamento che intende proteggere.

L'approccio di van Lamsweerde (2004) distingue goal, anti-goal, anti-requirement e vulnerabilità. La decomposizione dell'anti-model permette di rappresentare se un obiettivo ostile dipenda da una capacità dell'attaccante o da una debolezza dell'ambiente o del sistema. L'output dell'analisi non è quindi una lista piatta di categorie, ma una struttura di ragionamento.

Nel framework di Haley et al. (2008), i security requirements sono vincoli posti su specifici functional requirements. Un outer argument esprime le premesse necessarie affinché il requisito sia soddisfatto; inner arguments sottopongono tali premesse a grounds, warrants, rebuttals e trust assumptions. Quando l'argomento fallisce, il risultato non è necessariamente un nuovo "threat": può emergere un requisito infeasible, una conoscenza di contesto mancante o la necessità di introdurre nuove funzioni e nuovi requisiti.

Hatebur et al. (2006) rafforzano la distinzione fra problema e soluzione separando security problem frame, requisito concretizzato, principio di soluzione, protocollo, meccanismo e specifica. Le assunzioni sul dominio e sulle capacità dell'attaccante condizionano l'adeguatezza del meccanismo. Un requisito di sicurezza non può quindi essere ridotto alla semplice etichetta di una categoria di minaccia.

Queste fonti motivano una separazione che sarà utilizzata in tutta la tesi: la metodologia di analisi deve poter conservare la propria semantica specifica – scenario, anti-goal, failure mode, proprietà violata o categoria di minaccia – mentre il *Security Requirement* appartiene alla documentazione del prodotto e deve esprimere un'obbligazione comprensibile anche al di fuori della metodologia che l'ha originata. Il Capitolo 5 formalizzerà il passaggio dal finding accettato al requisito; nel background interessa stabilire che la letteratura tratta già minaccia, mitigazione, requisito, assunzione e prova di soddisfazione come artefatti concettualmente distinti.

Un ulteriore punto è la verificabilità. La presenza di un requisito di sicurezza non implica la sua soddisfazione. Haley et al. (2008) separano il requisito dagli argomenti che ne giustificano la soddisfazione; Whittle et al. (2008) distinguono specifica dell'attacco, comportamento composto ed evidenza ottenuta eseguendo trace note; Lonetti et al. (2023) separano modello di sicurezza, abstract test, concretizzazione e osservazione sul sistema reale. Questa distinzione permette di evitare che la generazione di un requisito sia interpretata come prova della sicurezza dell'implementazione.

2.4 Traceability, provenance e baseline

Se l'analisi parte dalla documentazione e produce nuovi artefatti, la possibilità di ricostruire la loro origine diventa parte del problema. La requirements traceability nasce storicamente dalla necessità di descrivere e recuperare la vita di un requisito e le sue relazioni con fonti e artefatti successivi (Gotel and Finkelstein, 1994). La disponibilità di repository, issue tracker e strumenti collaborativi non elimina automaticamente questa difficoltà.

La systematic review di Mucha et al. (2024) sulla pre-requirements-specification traceability mantiene centrali provenance, responsabilità, impact analysis e manutenzione e segnala versioning, trust, adaptability e industrial evaluation come problemi ancora aperti. Lo studio empirico di Ruiz et al. (2023), basato su survey e interviste con professionisti, mostra che nel campione analizzato la traceability rimane in larga parte manuale e che il suo valore dipende fortemente dall'integrazione nei processi, dalla collaborazione e dalla visibilità del beneficio. L'automazione è desiderata per le attività ripetitive, ma la verifica umana rimane rilevante.

Großer et al. (2022) aggiungono un elemento particolarmente importante per un processo documentation-driven: i documenti possono essere considerati viste su insiemi di requisiti e le relazioni possono avere semantica a livello di documento, requirement set e singolo requisito. Ridurre questa struttura a link non tipizzati fra nodi isolati può perdere informazione. La loro proof of concept mostra che alcune dipendenze possono essere formalizzate e controllate su grafo, ma non costituisce una valutazione completa di precisione, recall o efficacia industriale.

Per il resto della tesi è utile distinguere **traceability** e **provenance**. La traceability descrive relazioni navigabili fra artefatti: documento-requisito, requisito-modello, modello-finding, requisito-test. La provenance aggiunge le condizioni di origine: quale versione della fonte è stata utilizzata, quale trasformazione o regola ha prodotto un candidato, quali informazioni sono state selezionate o escluse, chi ha corretto o accettato il risultato e a quale baseline l'artefatto si riferisce.

Questa distinzione diventa ancora più importante quando le relazioni non vengono dichiarate direttamente da un autore ma proposte automaticamente. Lo studio di Wang et al. (2024) sul requirement-to-code traceability link recovery mostra che ranking e performance dei metodi cambiano con dataset, threshold, classifier e strategia di training; un buon ranking relativo non garantisce utilità operativa assoluta. Hey et al. (2024) mostrano inoltre che una fase di preprocessing o classificazione modifica quali frammenti raggiungono il recupero dei link: un filtro può migliorare una metrica e contemporaneamente ridurre il recall per una classe rara.

Di conseguenza, un link generato automaticamente deve essere interpretato come

risultato di una pipeline specifica. Occorre poter distinguere il candidate link dalla relazione revisionata e dalla eventuale ground truth utilizzata nella valutazione. Lo stesso principio vale per qualsiasi derivazione documentazione-modello o modello-finding.

La traceability permette inoltre di affrontare il change impact soltanto se le relazioni restano versionate e semanticamente interpretabili. Sapere che due file sono collegati non basta a stabilire che una modifica del primo invalidi il secondo. Occorre conoscere quale informazione sia stata effettivamente utilizzata, quale derivazione dipenda da essa e quale stato di revisione avesse il risultato. La nozione di **baseline** è quindi il punto di riferimento temporale che permette di affermare: “questo finding è stato prodotto utilizzando questa versione delle fonti, questo modello e questa configurazione metodologica”. Il problema della rivalutazione dopo il cambiamento, che verrà affrontato dalla RQ4, nasce da questa premessa.

2.5 Dal model-driven al documentation-driven

Gli approcci model-driven assumono che una rappresentazione esplicita e sufficientemente strutturata del sistema sia disponibile come input per trasformazioni, analisi o generazione di artefatti. La forza del paradigma è evidente: regole e verifiche possono operare su tipi e relazioni definite. Rispetto al problema della tesi, il limite è che la costruzione del modello viene spesso trattata come un’attività precedente.

La review sull’automated threat modelling di Granata and Rak (2024) rende questo confine particolarmente chiaro. L’automazione parte tipicamente dopo che il sistema è stato rappresentato come DFD, grafo tipizzato, modello derivato dal codice o inventory di asset. Da quel punto cataloghi, label, proprietà, relazioni e regole possono attivare threat candidate. Il numero di candidate generati non è però una misura di completezza o correttezza: tool differenti associano minacce a unità di modello e granularità differenti.

Il model-based security testing presenta un confine analogo a valle. Un modello arricchito con proprietà o attacchi consente la generazione di abstract test, ma l’esecuzione richiede ulteriori artefatti e trasformazioni (Lonetti et al., 2023). L’evidenza dipende dal modello, dal criterio di selezione, dalla concretizzazione, dall’adapter, dall’ambiente e dall’oracle. La pipeline è precisamente verificabile perché i suoi input e le sue trasformazioni sono espliciti.

Il problema documentation-driven si colloca a monte. La systematic review di Umar and Lano (2024) sulle tecniche automatizzate di Requirements Engineering mostra che il linguaggio naturale è l’input dominante e che gli strumenti possono produrre UML, requisiti strutturati, classificazioni, quality findings, link e altri candidate artifact. La

maggior parte degli strumenti esaminati è tuttavia semi-automatica e one-shot; anche trasformazioni definite fully automated vengono inserite in processi nei quali gli output sono verificati da persone. La review identifica come esigenze aperte la traceability verso le source statements, l'evoluzione dei modelli, l'human-in-the-loop governance, benchmark condivisi e validazione industriale.

Il mapping study di Souza et al. (2019) sul passaggio requirements-to-architecture arriva a una conclusione analoga. Requisiti testuali, goal model, feature model, problem frame e altre rappresentazioni vengono trasformati in candidate architectural views, ma le regole sono specifiche della rappresentazione e le decisioni su alternative e trade-off dipendono fortemente dalla conoscenza dell'architetto. Una struttura derivata non costituisce di per sè l'unica architettura corretta nè dimostra la soddisfazione dei requisiti.

In questa tesi, **documentation-driven** non significa quindi “analizzare direttamente testo grezzo senza costruire alcun modello”. Significa che la documentazione governata rimane la fonte autorevole della conoscenza sul progetto e che le rappresentazioni analitiche devono conservare un legame esplicito con tale fonte. Il passaggio documentazione-modello deve essere governato, validato e mantenuto nel tempo. La letteratura supporta la possibilità di produrre candidate artifact da documentazione, ma non stabilisce una trasformazione automatica, completa e semanticamente affidabile da documentazione eterogenea a un unico modello neutrale.

Questa differenza permette anche di separare due domande che altrimenti verrebbero confuse. La prima è: “una tecnica sa produrre threat candidate quando riceve un modello adatto?”. La seconda è: “la documentazione ordinaria del progetto contiene e mantiene la conoscenza necessaria per costruire quel modello in modo affidabile?”. La letteratura sull'automated threat modelling risponde soprattutto alla prima; DDTA concentra il proprio problema di ricerca sulla seconda e sul collegamento stabile fra le due.

2.6 Automazione e trasformazioni assistite da LLM

L'automazione può rendere ripetibili alcune trasformazioni e ridurre lavoro manuale, ma il termine “automatico” non fornisce da solo una garanzia sulla qualità dell'artefatto prodotto. Un parser che verifica una notazione, una regola che attiva un candidate threat e un modello linguistico che genera un diagramma svolgono funzioni diverse e devono essere valutati con criteri diversi.

Ferrari et al. (2024) studiano la generazione con GPT-3.5 di sequence diagram UML astratti a partire da requisiti realistici e loro varianti. Gli output ottengono risultati forti per aderenza alla notazione, comprensibilità e terminologia, ma la correttezza non supera

in modo significativo il valore neutro e la completezza rimane imperfetta. Diagrammi visivamente plausibili possono omettere condizioni, nascondere contraddizioni, gestire male numeri e tempi, assegnare azioni al componente sbagliato o introdurre termini non supportati dalla fonte. La qualità sintattica non è quindi una proxy sufficiente della fedeltà semantica.

Garaccione et al. (2025) confrontano GPT-4o, DeepSeek v3, Gemini 2.5 Pro e Qwen 3 nella generazione di class diagram da esercizi testuali e separano syntax error, semantic error e completeness rispetto a soluzioni di riferimento. Un output parser-compatible può comunque contenere associazioni, direzioni, generalizzazioni e molteplicità errate. Lo studio rafforza la necessità di separare almeno validità sintattica, copertura della fonte e correttezza strutturale-semantica.

Abualhaija et al. (2025) mostrano un problema complementare nella generazione assistita di requisiti regolatori. Nel loro caso, una quota elevata degli output prodotti può essere corretta o parzialmente corretta mentre la copertura dell'insieme di reference requirements rimane bassa e i riferimenti possono essere incompleti o misgrounded. Il dominio legale rimane fuori dall'implementazione della tesi, ma l'evidenza è metodologicamente importante: **correttezza locale, coverage e groundedness sono dimensioni indipendenti.**

Il quarto confine è temporale. Angermeir et al. (2026) analizzano la riproducibilità di studi di software engineering basati su LLM commerciali. Dei package inizialmente ritenuti sufficientemente completi per una riproduzione, solo pochi risultano effettivamente eseguibili; nessuno dei cinque studi eseguiti viene riprodotto completamente. Oltre ai problemi generali di artefatti, dipendenze e codice, la deprecazione dei modelli commerciali introduce un vincolo specifico. Un model identifier, un repository pubblico o un artifact badge non bastano a stabilire che un risultato sia ancora eseguibile o comparabile.

Da queste fonti emergono quattro gate indipendenti per un artefatto generato o misurato con LLM:

1. **validità sintattica e leggibilità:** l'artefatto può essere interpretato o processato;
2. **fedeltà semantica:** struttura e comportamento sono coerenti con la fonte;
3. **coverage e grounding:** è noto quale parte dell'evidenza sia rappresentata e su quali fonti si fonda;
4. **riproducibilità temporale:** input, configurazione, ambiente, output e interventi possono essere ricostruiti e confrontati.

Nessuno dei quattro gate sostituisce gli altri. Un modello corretto sintatticamente può essere semanticamente errato; un output corretto può coprire soltanto una piccola

parte della fonte; un esperimento ben documentato può diventare non eseguibile dopo il cambiamento di dipendenze o API. Per questo, nel resto della tesi, gli output automatici vengono trattati come **candidate artifacts** finché non esiste l'evidenza richiesta per la loro accettazione. Si tratta di una scelta che verrà formalizzata nel Capitolo 5, ma che è già motivata dai failure mode osservati nella letteratura.

2.7 Viewpoint metodologici e STRIDE-AI come caso asset-centered

Le sezioni precedenti mostrano che le metodologie di threat analysis non differiscono soltanto per la tassonomia delle minacce. Differiscono anche per la conoscenza che richiedono e per il punto di vista attraverso il quale osservano il sistema. Questa eterogeneità è il presupposto del problema multi-metodo affrontato nella tesi.

STRIDE-AI è particolarmente utile come riferimento di background perché rende esplicito un **asset-centered viewpoint** applicato a sistemi AI/ML. Mauri and Damiani (2022) presentano STRIDE-AI come estensione di STRIDE adattata al dominio AI/ML e orientata alle proprietà di sicurezza. L'obiettivo non è soltanto applicare le sei categorie STRIDE a componenti convenzionali, ma partire dagli asset generati e utilizzati lungo il ciclo di vita ML, analizzarne le possibili failure mode e collegare tali failure mode a proprietà di sicurezza e minacce.

2.7.1 Il ciclo di vita e gli asset in STRIDE-AI

Il riferimento di Mauri e Damiani utilizza un ciclo di vita ML iterativo che parte dai requirements e attraversa Data Management, Model Learning, Model Tuning, Model Deployment e Model Maintenance. La rappresentazione è significativa per due ragioni. Primo, l'analisi include asset che esistono molto prima del deployment, come requisiti, raw data, labeled data, validation data, algoritmi e hyper-parameters. Secondo, gli autori assumono esplicitamente che il sistema continui a evolvere e che l'identificazione degli asset debba essere aggiornata insieme alla soluzione AI/ML (Mauri and Damiani, 2022).

Gli asset vengono organizzati in sei macro-categorie: **Data, Models, Actors, Processes, Tools** e **Artefacts**. Le tabelle del paper approfondiscono soprattutto Data, Models e Artefacts. Fra i data asset compaiono, ad esempio, requisiti, raw data, pre-processed data, labeled data, validation data, augmented data, held-out test cases e inferences. Fra i model asset compaiono algoritmi di preprocessing, hyper-parameters, learning algorithms, model parameters, trained model e deployed model. Fra gli artefatti figurano model architecture, hardware design, data and metadata schemata e learned indexes (Mauri and Damiani, 2022).

La scelta è importante per la tesi perché mostra che “asset” non è sinonimo di “componente software”. Un requisito, un training set, un hyper-parameter o una schema definition possono essere oggetti rilevanti per l’analisi pur appartenendo a livelli diversi del progetto. Un core multi-metodo non può quindi assumere che tutti gli elementi analizzabili siano processi, data store o data flow.

2.7.2 FMEA come passaggio dagli asset alle failure mode

STRIDE-AI utilizza la Failure Mode and Effects Analysis (FMEA) per strutturare il passaggio dall’asset al possibile fallimento. La procedura descritta dagli autori comprende quattro attività: costruire una lista di funzioni per ciascun asset; specificare prerequisiti e dipendenze fra funzioni; identificare difetti o failure mode con cause ed effetti; utilizzare una metodologia di threat modeling per mappare le failure mode alle minacce (Mauri and Damiani, 2022).

Le domande facilitatrici rendono esplicito il tipo di conoscenza richiesto. Per una funzione occorre sapere quale sia lo scopo dell’asset, cosa debba e non debba fare, quali funzioni avvengano alle interfacce e quale standard di performance sia atteso. Per le failure mode si chiede in quale modo l’asset possa fallire, eseguire una funzione indesiderata, essere abusato o produrre problemi alle interfacce. Per gli effetti si valuta la conseguenza locale, di sistema e per l’utente, includendo potenziali danni e violazioni (Mauri and Damiani, 2022).

Questo passaggio mostra una differenza sostanziale rispetto a un semplice catalog lookup. La minaccia non viene dedotta soltanto dal tipo tecnico dell’asset. Il metodo richiede conoscenza della funzione, delle dipendenze, del comportamento atteso e del modo in cui tale comportamento può fallire. Per un sistema documentation-driven, queste informazioni devono quindi essere disponibili nella documentazione o essere introdotte esplicitamente come conoscenza aggiuntiva e revisionata.

2.7.3 Dalle proprietà alle minacce

Il paper descrive un processo di threat modeling in cinque passi: identificazione degli obiettivi di sicurezza, survey degli asset e delle interconnessioni, decomposition degli asset rilevanti, threat identification e vulnerability identification. STRIDE-AI fornisce una specializzazione ML-specifica delle proprietà CIA³-R e le collega alle sei categorie STRIDE (Mauri and Damiani, 2022).

Gli autori adattano anche la definizione delle proprietà al dominio ML. L’authenticity riguarda la possibilità di attribuire in modo verificabile l’output al modello; l’integrity riguarda la protezione delle informazioni usate o generate nel ciclo di vita; la non-repudiation riguarda l’impossibilità di negare che un output sia stato generato dal

Tabella 2.2: Mappatura fra proprietà di sicurezza e categorie STRIDE in STRIDE-AI, riformulata da Mauri and Damiani (2022).

Proprietà	Categoria di minaccia
Authenticity	Spoofing
Integrity	Tampering
Non-repudiation	Repudiation
Confidentiality	Information Disclosure
Availability	Denial of Service
Authorization	Elevation of Privilege

modello; la confidentiality riguarda l’assenza di disclosure oltre a input e output previsti; l’availability è collegata alla capacità del modello di produrre output utili rispetto allo scopo; l’authorization limita input e output alle parti autorizzate (Mauri and Damiani, 2022).

Le tabelle di mapping nel paper mostrano che asset differenti richiedono profili differenti. I labeled data vengono collegati principalmente ad Authenticity e Integrity e quindi a Spoofing e Tampering; un deployed model può coinvolgere tutte le proprietà e tutte le categorie STRIDE. Gli esempi includono poisoning, model extraction, membership inference, image-scaling attacks, label flipping, inference eavesdropping e altri attacchi specifici del ciclo di vita ML (Mauri and Damiani, 2022). Il valore concettuale per la tesi non è l’elenco in sè, ma il fatto che il metodo conserva una catena esplicita:

asset → funzione/failure mode → proprietà a rischio → categoria di minaccia → condizioni di vulnerabilità.

2.7.4 Il caso TOREADOR e il limite dell’evidenza

Mauri e Damiani illustrano STRIDE-AI con un caso reale proveniente dal progetto europeo TOREADOR. Il sistema riguarda predictive maintenance e ottimizzazione energetica in impianti fotovoltaici. L’architettura comprende data logger, infrastruttura LIGHT, preprocessing, connettori verso la piattaforma TOREADOR, training e inferenza del modello e restituzione delle predizioni agli operatori (Mauri and Damiani, 2022).

Nel caso di studio gli autori identificano asset data e model lungo l’architettura. Per il training data stream selezionano Integrity e Authenticity, derivando Tampering e Spoofing; per il trained model considerano Integrity e Availability, derivando Tampering e DoS. Le threat tree mostrano poi condizioni concrete: impersonation fra LIGHT e TOREADOR, injection o modification dei dati, label flipping e interferenze sull’esecuzione del modello. Alcuni rami possono essere eliminati grazie a dettagli dell’architettura, come la firma dei dati da parte dei logger, mentre altri rimangono

possibili perché le label vengono introdotte successivamente (Mauri and Damiani, 2022). Questo passaggio è particolarmente rilevante: la validità del finding dipende da informazioni architetturali e operative specifiche, non dalla sola categoria STRIDE.

Gli autori utilizzano DREAD come rating preliminare per tre threat considerate nel caso, ma ricordano che DREAD è soggetto a inconsistenza e non è raccomandato come unico strumento di prioritizzazione. Il caso TOREADOR va quindi interpretato come dimostrazione della procedura, non come benchmark controllato dell'efficacia comparativa di STRIDE-AI.

La sezione di discussione del paper contiene infine un limite direttamente collegato alla manutenzione: gli autori osservano che non è realistico enumerare in un singolo passaggio tutti i fenomeni di failure di un sistema ML e che le failure mode devono essere aggiornate con nuove informazioni sul sistema e con l'evoluzione delle conoscenze su vulnerabilità e attacchi (Mauri and Damiani, 2022). Questa posizione rafforza il background della RQ4: anche una metodologia strutturata richiede un rapporto esplicito fra analisi, stato del sistema e cambiamento.

2.7.5 Perchè STRIDE e STRIDE-AI sono utili al problema multi-metodo

La tesi utilizza STRIDE e STRIDE-AI come due implementazioni concrete per verificare il boundary di plugin e overlay. La scelta non implica che DDTA sia una generalizzazione di STRIDE, nè che due metodi siano sufficienti a dimostrare supporto universale.

STRIDE fornisce un caso vicino agli approcci software/component-centered frequenti negli strumenti automatizzati. STRIDE-AI, pur derivando storicamente da STRIDE, introduce un centro analitico più ricco: asset lungo il ciclo di vita ML, funzioni, failure mode, proprietà ML-specifiche, knowledge about attacker capability e condizioni di vulnerabilità. Il test scientificamente interessante non è quindi verificare se i due metodi condividano le sei etichette STRIDE, ma se possano consumare una stessa rappresentazione canonica della conoscenza di sistema senza obbligare il core a incorporare la semantica privata di uno dei due.

L'asset-centered viewpoint di STRIDE-AI offre un criterio concreto per verificare questo confine. Se la rappresentazione comune fosse modellata soltanto attorno a componenti e data flow, potrebbe non contenere informazioni sufficienti su training data, model parameters, hyper-parameters, tool, artefatti o funzioni degli asset. Se, al contrario, il core incorporasse direttamente tutte le categorie e le regole di STRIDE-AI, cesserebbe di essere metodologicamente neutrale. La valutazione multi-metodo della tesi si colloca precisamente fra questi due estremi.

2.8 Sintesi del background

Il background costruito dalle fonti porta a sei premesse per i capitoli successivi.

La prima è che **la threat analysis può iniziare prima dell'implementazione**. Requisiti, scenari, goal, contesti, architetture e altri modelli sono utilizzati da numerose tecniche già nelle fasi di requirements, architecture e design (Tuma et al., 2018). Anticipare l'analisi alla fase documentale è quindi un problema scientificamente plausibile.

La seconda è che **documentazione e modello non sono sinonimi**. La documentazione può contenere l'evidenza iniziale, ma le metodologie richiedono normalmente strutture interpretate: scenari, goal, domini, fenomeni, componenti, flussi, asset, funzioni, proprietà e relazioni. La trasformazione dalla documentazione eterogenea a una rappresentazione analizzabile è pertanto un problema separato e non risolto automaticamente dalla letteratura.

La terza è che **non emerge una singola rappresentazione sufficiente per ogni metodologia**. Scenario-based, goal-oriented, software-centric, requirements-oriented e asset-centered approaches portano in primo piano informazioni differenti. STRIDE-AI rende particolarmente esplicito il caso asset-centered, nel quale l'analisi attraversa il ciclo di vita ML e richiede conoscenza su data, model, actors, processes, tools e artefacts (Mauri and Damiani, 2022). Il problema della tesi non può quindi essere ridotto alla scelta di una notazione di threat modeling esistente.

La quarta è che **finding e Security Requirement sono artefatti distinti**. Le fonti su Security Requirements Engineering separano problema, minaccia, mitigazione, requisito, assunzione e prova di soddisfazione (Sindre and Opdahl, 2005; Haley et al., 2008; Hatebur et al., 2006). Una metodologia può produrre evidenza per giustificare un requisito senza imporre che la tassonomia del metodo diventi parte del requisito stesso.

La quinta è che **automazione e accettazione devono rimanere separate**. Le review sull'automated RE e gli studi LLM mostrano che un artefatto può essere leggibile ma incompleto, sintatticamente valido ma semanticamente errato, localmente corretto ma con bassa coverage oppure non più riproducibile nel tempo (Umar and Lano, 2024; Ferrari et al., 2024; Garaccione et al., 2025; Abualhaija et al., 2025; Angermeir et al., 2026). La qualità deve quindi essere misurata rispetto alla specifica trasformazione e all'evidenza che essa dichiara di produrre.

La sesta è che **traceability, provenance e change awareness sono parte della qualità analitica**. Un finding non riconducibile alle fonti, alla baseline, alla trasformazione e al metodo che lo hanno prodotto è difficile da rivalutare dopo un cambiamento (Mucha et al., 2024; Großer et al., 2022). La stessa considerazione vale per un *Security Requirement* derivato dal finding.

Queste premesse non costituiscono ancora la soluzione DDTA. Definiscono invece il problema che il Capitolo 3 confronterà con lo stato dell'arte: come definire una documentazione governata *portable-by-construction*, sufficientemente esplicita da alimentare una rappresentazione analizzabile senza legarla a un solo metodo, permettere a view-point metodologici differenti di operare sulla stessa conoscenza di sistema, preservare provenance e revisione e riportare i risultati accettati nella documentazione sotto forma di requisiti di sicurezza. La migrazione da documentazione generica o legacy verso tale forma resta fuori dal perimetro della tesi corrente. Il research gap verrà formulato soltanto dopo questo confronto.

Bibliografia

- O. C. Z. Gotel and A. C. W. Finkelstein. An Analysis of the Requirements Traceability Problem. In *Proceedings of the First International Conference on Requirements Engineering*, pages 94–101, 1994. doi:10.1109/ICRE.1994.292398.
- G. Sindre and A. L. Opdahl. Eliciting Security Requirements with Misuse Cases. *Requirements Engineering*, 10(1):34–44, 2005. doi:10.1007/s00766-004-0194-4.
- A. van Lamsweerde. Elaborating Security Requirements by Construction of Intentional Anti-Models. In *Proceedings of the 26th International Conference on Software Engineering*, pages 148–157, 2004. doi:10.1109/ICSE.2004.1317437.
- J. Mucha, A. Kaufmann, and D. Riehle. A systematic literature review of pre-requirements specification traceability. *Requirements Engineering*, 29(2):119–141, 2024. doi:10.1007/s00766-023-00412-z.
- M. Ruiz, J. Y. Hu, and F. Dalpiaz. Why don’t we trace? A study on the barriers to software traceability in practice. *Requirements Engineering*, 28(4):619–637, 2023. doi:10.1007/s00766-023-00408-9.
- K. Großer, V. Riediger, and J. Jürjens. Requirements document relations: A reuse perspective on traceability through standards. *Software and Systems Modeling*, 21:2133–2169, 2022. doi:10.1007/s10270-021-00958-y.
- B. Wang, Z. Zou, H. Wan, Y. Li, Y. Deng, and X. Li. An empirical study on the state-of-the-art methods for requirement-to-code traceability link recovery. *Journal of King Saud University - Computer and Information Sciences*, 36(6):102118, 2024. doi:10.1016/j.jksuci.2024.102118.
- T. Hey, J. Keim, and S. Corallo. Requirements Classification for Traceability Link Recovery. In *2024 IEEE 32nd International Requirements Engineering Conference (RE)*, pages 155–167, 2024. doi:10.1109/RE59067.2024.00024.
- K. Tuma, G. Calikli, and R. Scandariato. Threat analysis of software systems: A systematic literature review. *Journal of Systems and Software*, 144:275–294, 2018. doi:10.1016/j.jss.2018.06.073.

- C. B. Haley, R. Laney, J. D. Moffett, and B. Nuseibeh. Security Requirements Engineering: A Framework for Representation and Analysis. *IEEE Transactions on Software Engineering*, 34(1):133–153, 2008. doi:10.1109/TSE.2007.70754.
- J. Whittle, D. Wijesekera, and M. Hartong. Executable Misuse Cases for Modeling Security Concerns. In *Proceedings of the 30th International Conference on Software Engineering*, pages 121–130, 2008. doi:10.1145/1368088.1368106.
- D. Hatebur, M. Heisel, and H. Schmidt. Security Engineering Using Problem Frames. In *Emerging Trends in Information and Communication Security*, LNCS 3995, pages 238–253. Springer, 2006. doi:10.1007/11766155_17.
- W. B. Mbaka, X. Zhang, Y. Wang, T. Li, F. Massacci, and K. Tuma. Assessing the usefulness of Data Flow Diagrams for validating security threats. *Computers & Security*, 156:104498, 2025. doi:10.1016/j.cose.2025.104498.
- D. Granata and M. Rak. Systematic analysis of automated threat modelling techniques: Comparison of open-source tools. *Software Quality Journal*, 32:125–161, 2024. doi:10.1007/s11219-023-09634-4.
- F. Lonetti, A. Bertolino, and F. Di Giandomenico. Model-based security testing in IoT systems: A Rapid Review. *Information and Software Technology*, 164:107326, 2023. doi:10.1016/j.infsof.2023.107326.
- M. A. Umar and K. Lano. Advances in automated support for requirements engineering: a systematic literature review. *Requirements Engineering*, 29:177–207, 2024. doi:10.1007/s00766-023-00411-0.
- E. Souza, A. Moreira, and M. Goulão. Deriving architectural models from requirements specifications: A systematic mapping study. *Information and Software Technology*, 109:26–39, 2019. doi:10.1016/j.infsof.2019.01.004.
- A. Ferrari, S. Abualhaija, and C. Arora. Model Generation with LLMs: From Requirements to UML Sequence Diagrams. In *2024 IEEE 32nd International Requirements Engineering Conference Workshops*, pages 291–300, 2024. doi:10.1109/REW61692.2024.00044.
- G. Garaccione, D. M. Calabrese, R. Coppola, and L. Ardito. A comparison of different Large Language Models for the generation of UML class diagrams. In *2025 ACM/IEEE 28th MODELS Companion*, pages 541–545, 2025. doi:10.1109/MODELS-C68889.2025.00078.
- S. Abualhaija, M. Ceci, N. Sannier, D. Bianculli, S. Lannier, M. Siclari, O. Voordeckers, and S. Tosza. LLM-assisted Extraction of Regulatory Requirements: A Case Study on the GDPR. In *2025 IEEE 33rd International Requirements Engineering Conference*, pages 142–154, 2025. doi:10.1109/RE63999.2025.00023.

- F. Angermeir, M. Amougou, M. Kreitz, A. Bauer, M. Linhuber, D. Fucci, F. Moyón C., D. Mendez, and T. Gorschek. Reflections on the Reproducibility of Commercial LLM Performance in Empirical Software Engineering Studies. In *Proceedings of the 48th IEEE/ACM International Conference on Software Engineering*, 2026. doi:10.1145/3744916.3773207.
- L. Mauri and E. Damiani. Modeling Threats to AI-ML Systems Using STRIDE. *Sensors*, 22(17):6662, 2022. doi:10.3390/s22176662.